

Testkonzept

Was wird getestet, und was wird nicht getestet?

- All unsere Code-Artefakte (Klassen, Methoden), die im Klassendiagramm stehen

Begründung: Damit Falscheingaben erkannt werden und übertriebene Eingaben erkannt werden.

Wie wird getestet?

Begründung: Da dieses Projekt sehr klein ist, verwenden wir für das Testing vor allem automatische Tests. Die System Tests führen wir manuell durch, indem wir das Spiel dreimal durchspielen (X-gewinnt, O-gewinnt, Unentschieden).

Diese Tests werden in 3 Kategorien untergeordnet:

1. Unit-Tests: Getestet werden alleinstehende Klassen und alle ihre public Methoden.
2. Integrationstests: Getestet werden die Schnittstellen der Klassen untereinander.
3. System Tests: Getestet wird das ganze fertige System, als wäre es normal angewendet.

Für die Tests verwenden wir die Test Library JUnit.

Wer wird die Tests entwickeln und ausführen?

1. Für die Tests der Klassen, sollte die Person, die gerade eine Methode schreibt, auch gleich einen Test für diese Methode schreiben und durchführen. Dann hat man direkt eigenes Feedback und kann die Methode verbessern.
2. Die Integrationstests machen wir direkt mit den Komponenten-Tests zusammen, wenn es gerade Sinn macht bzw. wenn eine Methode auf eine andere Klasse zugreift, dann wird auch direkt die Schnittstelle getestet.
3. Die System Tests machen wir zusammen, wenn das Programm fertig ist bzw. in einem Zustand ist, wo es läuft.

Begründung: Uns scheint diese Herangehensweise am effizientesten und so sollte auch jeder Mal zum Tests-Schreiben kommen.

Wann werde die Tests entwickelt und ausgeführt?

Test Timeline: Komponenten-Tests → Integrationstests → System Tests

1. Komponenten-Tests: Während dem Erstellen Methoden
2. Integrationstests: Wenn mehrere Klassen existieren und miteinander kommunizieren
3. System Tests: Wenn alle Klassen & Methoden existieren und das Programm läuft

Begründung: Wenn wir während dem Programmieren laufen testen, überprüfen wir konstant unseren Code und haben eine bessere Code-Qualität. Und einige Tests können wir erst mit dem fertigen Programm durchführen.

Welche Tests werden entwickelt und ausgeführt?

Wir testen nicht jede Getter und Setter Methode.

Wir testen den effektiven Output in der Console nicht.

Wir testen die drei Spielausgänge indem wir das Spiel 3x manuell spielen – den Rest testen wir automatisiert.

TicTacToe

Wir testen...

...ob TicTacToe die anderen Klassen richtig instanziiert.

...ob es die richtigen Nachrichten über SendMessage ausgibt.

...ob es stetig Inputs aufnimmt vom User, bis das Spiel fertig ist.

Board

Wir testen...

...ob das Board und die Spielzüge richtig in der Konsole ausgegeben werden.

...ob das Board richtig neugestartet wird.

InputController

Wir testen...

...ob der InputController Inputs (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) erkennt.

...ob der InputController falsche Inputs (zB. Negative Zahlen, Strings) erkennt.

...ob der InputController übertriebene Inputs (zB. 1111111111) erkennt.

LanguageController

Wir testen...

...ob der LanguageController die Sprache richtig wechselt, falls der Spieler eine andere Sprache wählt.

...ob der LanguageController erkennt, dass eine gültige Sprache (Englisch 1, oder Deutsch 2) gewählt wurde.

...ob der LanguageController die richtigen Nachrichten ausgibt.

EndstateChecker

Wir testen...

...ob der EndStateChecker O-gewinnt (drei O in einer Reihe) erkennt.

...ob der EndStateChecker X-gewinnt (drei X in einer Reihe) erkennt.

...ob der EndStateChecker ein Unentschieden (volles Spielfeld und kein Win/Loss) erkennt.

...ob der EndStateChecker erkennt, dass das Spiel keinen Endzustand hat (leeres Spielfeld) und fortgesetzt wird.

ValidMoveChecker

Wir testen...

...ob der ValidMoveChecker ungültige Spielzüge (Zahlen $\neq$ 1-9, Strings, oder schon ein Spielstein auf dem gewählten Feld liegt) erkennt.

...ob der ValidMoveChecker gültige Spielzüge (Zahlen = 1-9 für leere Spielfelder) erkennt.